

# 1 cartonase - Megoldás és magyarázat

## 1.1 Probléma áttekintése

A feladat egy  $n$  elemű tömböt dolgoz fel három különböző esetre bontva, attól függően, hogy a  $c$  paraméter melyik értéket veszi fel. A megoldás lényege, hogy hatékonyan elemezzük a tömb elemeit prefixumok szerint, és meghatározzuk bizonyos speciális pozíciókat.

## 1.2 1. eset ( $c = 1$ ): Elemek számlálása

Az első részfeladatban meghatározzuk, hogy a  $poz$  pozíció előtti elemek közül hány darab kisebb értékű, mint a  $poz$  pozíción található elem.

**Key observation:** Mivel nem kell minden elemhez külön összehasonlítást végezni, elég végigiterálni az első  $poz$  pozícióig, és megszámolni az elemeket. A kód optimalizált megoldása:

```
for (i=1; i<=poz; i++)
    if (v[i] >= v[poz])
        break;
fout<<i-1<<"\n";
```

Ez valójában egy-lineáris keresést végez: megállunk, amikor egy elem eléri vagy meghaladja a célértéket, és az azt megelőző elemek száma adja a választ.

## 1.3 2. eset ( $c = 2$ ): Prefixum maximumok

A második részfeladatban megkeressük azokat a pozíciókat  $i$ , ahol a prefixum  $[1..i]$  maximuma pontosan  $i$ .

**Key observation:** Ha a maximum az  $i$ -edik pozícióban  $i$ , akkor a tömb tartalmazza az összes értéket 1-től  $i$ -ig legalább egyszer. Ez azért fontos, mert ellenőrzi, hogy a prefixum „teli-e” a  $[1..i]$  értékhalommal:

$$\max\{v[1], v[2], \dots, v[i]\} = i \iff \{1, 2, \dots, i\} \subseteq \{v[1], v[2], \dots, v[i]\}$$

A kód tehát folyamatosan frissíti a maximumot, és ellenőrzi, hogy mikor egyenlő az aktuális indexszel.

## 1.4 3. eset ( $c = 3$ ): Két legnagyobb elem vizsgálata

A harmadik részfeladatban két legnagyobb elemet tartunk nyilván: a legnagyobbat ( $\text{maxim}_1$ ) és a második legnagyobbat ( $\text{maxim}_2$ ). Azt a pozíciót keressük, ahol  $\text{maxim}_1 > i$  és  $\text{maxim}_2 \leq i$ .

**Key observation:** Ez a feltétel azt jelenti, hogy a prefixumban van olyan elem, amely nagyobb a pozíciónál, de minden más elem legfeljebb akkora, mint a pozíció. Másként fogalmazva: a prefixumból pontosan egy elem haladja meg az  $i$  értéket.

A kód implementációja hatékonyan frissíti a két maximumot minden lépésben:

```
if (v[i] > maxim1) {  
    maxim2 = maxim1;  
    maxim1 = v[i];  
} else if (v[i] > maxim2)  
    maxim2 = v[i];
```

Ez a  $O(n)$  időkomplexitású megoldás biztosítja, hogy minden pozícióhoz tudjuk a két legnagyobb elemet.

## 1.5 Összegzés

A megoldás időkomplexitása minden esetben  $O(n)$ , mivel a tömböt egyszer járjuk be. A térkomplexitás  $O(DIM)$  a frekvencia tömb miatt. A feladat lényege, hogy felismerjük a prefixum-alapú tulajdonságokat, és hatékonyan karbantartjuk a szükséges maximum értékeket.